

Bioinformatics: Finding Regulatory Motifs

Swakkhar Shatabda

Department of Computer Science and Engineering
BRAC University



Chapter 2, Bioinformatics Algorithms: An Active Learning Approach - I



Inspiring Excellence

Hidden Message Once Again!

- Gene regulation is the process used to control the timing, location and amount in which genes are expressed.
- The process can be complicated and is carried out by a variety of mechanisms, including through regulatory proteins and chemical modification of DNA. Gene regulation is key to the ability of an organism to respond to environmental changes.
- A **transcription factor** regulates a gene by binding to a specific short DNA interval called a **regulatory motif**, or **transcription factor binding site**.
- Transcription factors bind to either **enhancer** or **promoter** regions of DNA adjacent to the genes that they regulate based on recognizing specific DNA motifs.



Inspiring Excellence

Regulatory Motifs

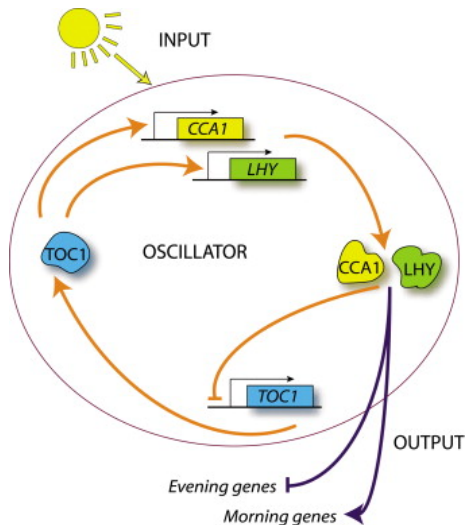
- Motifs are short recurring patterns.
- Transcription factors often regulate a group of genes that are involved in similar cellular processes.
- Thus, genes that contain the same motif in their upstream regions are likely to be related in their functions.
- In fact, many regulatory motifs are identified by analyzing the regions upstream of genes known to have similar functions.
- The life of a bioinformatician would be easy if regulatory motifs were completely conserved, but the reality is more complex, as regulatory motifs may vary at some positions, **denegate**.



Inspiring Excellence

Circadian Clock

- Plant cell keeps track of day and night independently of other cells, and that just three plant genes, called LHY, CCA1, and TOC1, are the clock's master timekeepers.
- TOC1 promotes the expression of LHY and CCA1, whereas LHY and CCA1 repress the expression of TOC1, resulting in a negative feedback loop.
- CCA1 binds to AAAAAATCT in the upstream region of many genes regulated by CCA1



The evening element

- In 2000, Steve Kay used DNA arrays to determine which genes in the plant *Arabidopsis thaliana* are activated at different times of the day.
- He then extracted the upstream regions of nearly 500 genes that exhibited circadian behavior and looked for frequently appearing patterns in their upstream regions.
- If you concatenated these upstream regions into a single string, you would find that AAAATATCT is a surprisingly frequent word, appearing 46 times.
- After he mutated the evening element in the upstream region of one gene, the gene no longer exhibited circadian behavior.
- Not all motifs are as conserved as the evening element.



Inspiring Excellence

Immunity genes in a Fly

- If we infect a fly with a bacterium, the fly will switch on its immunity genes to fight the infection.
- The genes with elevated expression levels after the infection are likely to be immunity genes.
- These genes have 12-mers similar to TCGGGGATTTC in upstream regions, binding site of a transcription factor called NF- κ B that activates various immunity genes in flies.

1	T	C	G	G	G	G	g	T	T	T	t	t
2	c	C	G	G	t	G	A	c	T	T	a	C
3	a	C	G	G	G	G	A	T	T	T	t	C
4	T	t	G	G	G	G	A	c	T	T	t	t
5	a	a	G	G	G	G	A	c	T	T	C	C
6	T	t	G	G	G	G	A	c	T	T	C	C
7	T	C	G	G	G	G	A	T	T	c	a	t
8	T	C	G	G	G	G	A	T	T	c	C	t
9	T	a	G	G	G	G	A	a	c	T	a	C
10	T	C	G	G	G	t	A	T	a	a	C	C

Another string finding problem

```
1 atgaccgggatactgataaaaaaaaaagggggggcggtacacattagataaacgtatgaagtacgttagactcggcgccgcccg
2 acccctatTTTTTgagcagatttagtgacctggaaaaaaaaatttgagtacaaaactttccgaataaaaaaaaaaggggggga
3 tgagtatccctgggatgacttaaaaaaaaggggggggtgctctcccgatttttgaatatgtaggatcattcgccaggggtccga
4 gctgagaattggatgaaaaaaaaaggggggggtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
5 tcccttttgcggtaatgtgccgggaggctgggtacgtagggaagccctaacggacttaataaaaaaaaaagggggggccttatag
6 gtcaatcatgttcttgtgaatggatttaaaaaaaaggggggggaccgcttggcgcacccaaattcagtggtggcgagcgcaa
7 cggttttggcccttggttagaggcccccgtaaaaaaaagggggggcaattatgagagagctaatctatcgcggtgcgtgttcat
8 aacttgagttaaaaaaaaaggggggggctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
9 ttggcccatggctaaaagcccaacttgacaaatggaagatagaatccttgcataaaaaaaaggggggggaccgaaagggaag
10 ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttaaaaaaaaggggggga
```



Inspiring Excellence

Frequent Words Problem?

```
1 atgaccgggatactgatAAAAAAAGGGGGGGggcgtacacattagataaacgtatgaagtacgttagactcggcgcgcgcg
2 acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaataAAAAAAAGGGGGGga
3 tgagtatccctgggatgacttAAAAAAAGGGGGGGtgctctcccgattTTTgaatatgtaggatcattcgccagggtccga
4 gctgagaattggatgAAAAAAAGGGGGGGtcacgcgaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
5 tccctTTTgcggaatgtgccgggaggctggttacgtagggaagccctaacggacttaataAAAAAAAGGGGGGGcttatag
6 gtcaatcatgttcttTgtgaatggatttAAAAAAAGGGGGGGgaccgcttgggcgacccaaattcagtggtggcgagcgcaa
7 cggTTTTggcccttgtagaggcccccgTAAAAAAAGGGGGGGcaattatgagagagctaattctatcgcgtgcgtgttcat
8 aacttgagttAAAAAAAGGGGGGGctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
9 ttggccattggctaaaagcccaacttgacaaatggaagatagaatccttgcatAAAAAAAGGGGGGGaccgaaagggaag
10 ctggtgagcaacgcagagattcttacctgcattagctcgcttcggggatctaatagcacgaagcttAAAAAAAGGGGGGga
```



Inspiring Excellence

Frequent Words with Mismatches Problem?

```
1 atgaccgggatactgatAgAAgAAAGGttGGGggcgtaacacattagataaaacgtatgaagtaacgttagactcggcgccgccc
2 acccctattttttgagcagatttagtacctggaaaaaatttgagtacaaaacttttccgaatacAAATAAAACGGCGGGa
3 tgagtatccctgggatgacttAAAAATAAGGAGtGGTgctctcccgatttttgaatatgtaggatcattcgccaggggtccga
4 gctgagaattggatgcAAAAAAAGGGattGtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
5 tcccttttgcggtaatgtgccgggaggtggttacgtagggaagccctaacggacttaataATAATAAGGaaGGGcttatag
6 gtcaatcatgttcttgtgaatggattAACAAATAAGGGctGGgaccgcttggcgcacccaaattcagtggtggcgagcgcaa
7 cggttttggcccttgttagaggccccgtATAAAACAGGAGGGccaattatgagagagctaatactatcgcggtcggtgttcat
8 aacttgagttAAAAAATAAGGAGccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
9 ttggcccatgtgctaaaagcccaactgacaaatggaagatagaatccttgcatActAAAAAGGAGcGGaccgaaagggaag
10 ctggtgagcaacgacagattcttacctgacattagctcgcttccgggatctaatagcacgaagcttActAAAAAGGAGcGGa
```

- Concatenating all the sequences into a single string is inadequate because it does not correctly model the biological problem of motif finding.
- A DnaA box is a pattern that clumps, or appears frequently, within a relatively short interval of the genome.
- In contrast, a regulatory motif is a pattern that appears at least once (with variation) in each of many different regions that are scattered throughout the genome.



Inspiring Excellence

A brute force algorithm

MOTIFENUMERATION(Dna, k, d)

$Patterns \leftarrow$ an empty set

for each k -mer $Pattern$ in Dna

for each k -mer $Pattern'$ differing from $Pattern$ by at most d mismatches

if $Pattern'$ appears in each string from Dna with at most d mismatches

 add $Pattern'$ to $Patterns$

remove duplicates from $Patterns$

return $Patterns$

- Each string length = n , number of strings = t , what will be the run time?



Inspiring Excellence

A better algorithm?

$$d(\text{Pattern}, \text{Text}) = \min_{\text{all } k\text{-mers } \text{Pattern}' \text{ in Text}} \text{HAMMINGDISTANCE}(\text{Pattern}, \text{Pattern}').$$



$$d(\text{GATTCTCA}, \text{gcaaaGACGCTGAccaa}) = 3.$$

$$d(\text{Pattern}, \text{Dna}) = \sum_{i=1}^t d(\text{Pattern}, \text{Dna}_i).$$

For example, for the strings *Dna* shown below, $d(\text{AAA}, \text{Dna}) = 1 + 1 + 2 + 0 + 1 = 5$.

	ttaccttAAC	1
	gATAtctgtc	1
<i>Dna</i>	ACGgcgttcg	2
	ccctAAAgag	0
	cgtcAGAggt	1

A better algorithm?

```
MEDIANSTRING(Dna, k)  
  distance  $\leftarrow \infty$   
  for each k-mer Pattern from AA . . . AA to TT . . . TT  
    if distance > d(Pattern, Dna)  
      distance  $\leftarrow d(\textit{Pattern}, \textit{Dna})$   
      Median  $\leftarrow \textit{Pattern}$   
  return Median
```

- Runtime? Comparison to the brute force?



Inspiring Excellence

Profile Matrix

```

1  T C G G G G g T T T t t
2  c C G G t G A c T T a C
3  a C G G G G A T T T t C
4  T t G G G G A c T T t t
5  a a G G G G A c T T C C
6  T t G G G G A c T T C C
7  T C G G G G A T T c a t
8  T C G G G G A T T c C t
9  T a G G G G A a c T a C
10 T C G G G t A T a a C C
    
```

Profile

A:	.2	.2	.0	.0	.0	.0	.9	.1	.1	.1	.3	.0
C:	.1	.6	.0	.0	.0	.0	.0	.4	.1	.2	.4	.6
G:	.0	.0	1	1	.9	.9	.1	.0	.0	.0	.0	.0
T:	.7	.2	.0	.0	.1	.1	.0	.5	.8	.7	.3	.4

$$\Pr(\text{ACGGGGATTACC}|\text{Profile}) = .2 \cdot .6 \cdot 1 \cdot 1 \cdot .9 \cdot .9 \cdot .9 \cdot .5 \cdot .8 \cdot .1 \cdot .4 \cdot .6 = 0.000839808$$

$$\Pr(\text{TCGGGGATTTC}|\text{Profile}) = 0.7 \cdot 0.6 \cdot 1.0 \cdot 1.0 \cdot 0.9 \cdot 0.9 \cdot 0.9 \cdot 0.5 \cdot 0.8 \cdot 0.7 \cdot 0.4 \cdot 0.6 \cdot 0.6 = 0.0205753,$$

A scoring function for the motifs

Motifs	T	C	G	G	G	G	g	T	T	T	t	t	
	c	C	G	G	t	G	A	c	T	T	a	C	
	a	C	G	G	G	G	A	T	T	T	t	C	
	T	t	G	G	G	G	A	c	T	T	t	t	
	a	a	G	G	G	G	A	c	T	T	C	C	
	T	t	G	G	G	G	A	c	T	T	C	C	
	T	C	G	G	G	G	A	T	T	c	a	t	
	T	C	G	G	G	G	A	T	T	c	C	t	
	T	a	G	G	G	G	A	a	c	T	a	C	
	T	C	G	G	G	t	A	T	a	a	C	C	
SCORE(Motifs)	3 + 4 + 0 + 0 + 1 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30												
COUNT(Motifs)	A:	2	2	0	0	0	0	9	1	1	1	3	0
	C:	1	6	0	0	0	0	0	4	1	2	4	6
	G:	0	0	10	10	9	9	1	0	0	0	0	0
	T:	7	2	0	0	1	1	0	5	8	7	3	4
PROFILE(Motifs)	A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
	C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
	G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
	T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4
CONSENSUS(Motifs)	T	C	G	G	G	G	A	T	T	T	C	C	



Entropy

- Entropy is a measure of the uncertainty of a probability distribution $(p_1, \dots, p_N)$, and is defined as:

$$H(p_1, \dots, p_N) = - \sum_{i=1}^N p_i \cdot \log_2(p_i)$$

- The entropy of the probability distribution (0.2, 0.6, 0.0, 0.2) corresponding to the second column is
 $-(0.2\log_2 0.2 + 0.6\log_2 0.6 + 0.0\log_2 0.0 + 0.2\log_2 0.2) \approx 1.371$
- The entropy of the more conserved final column (0.0, 0.6, 0.0, 0.4) is
 $-(0.0\log_2 0.0 + 0.6\log_2 0.6 + 0.0\log_2 0.0 + 0.4\log_2 0.4) \approx 0.971$
- The entropy of the very conserved 5th column (0.0, 0.0, 0.9, 0.1) is
 $-(0.0\log_2 0.0 + 0.0\log_2 0.0 + 0.9\log_2 0.9 + 0.1\log_2 0.1) \approx 0.467$.
- The entropy of a motif matrix is defined as the sum of the entropies of its columns.



A Greedy Algorithm

GREEDYMOTIFSEARCH(Dna, k, t)

$BestMotifs \leftarrow$ motif matrix formed by first k -mers in each string from Dna

for each k -mer $Motif$ in the first string from Dna

$Motif_1 \leftarrow Motif$

for $i = 2$ to t

form $Profile$ from motifs $Motif_1, \dots, Motif_{i-1}$

$Motif_i \leftarrow$ $Profile$ -most probable k -mer in the i -th string in Dna

$Motifs \leftarrow (Motif_1, \dots, Motif_t)$

if $SCORE(Motifs) < SCORE(BestMotifs)$

$BestMotifs \leftarrow Motifs$

return $BestMotifs$

- How good is this algorithm? It fails too.



Inspiring Excellence

Why greedy algorithm fails?

- Find the (4,1)-motif **ACGT** implanted in the following strings Dna.

tt**ACCT**taac

g**ATGT**ctgtc

acg**GCGT**tag

cccta**ACGA**g

cgtcag**AGGT**

A: 1 0 0 0

C: 0 1 1 0

G: 0 0 0 0

T: 0 0 0 1

- Laplacian Correction Needed.



Inspiring Excellence

Laplacian Correction

Motifs

T	A	A	C
G	T	C	T
A	C	T	A
A	G	G	T

COUNT(<i>Motifs</i>)	A:	2	1	1	1	PROFILE(<i>Motifs</i>)	2/4	1/4	1/4	1/4
	C:	0	1	1	1		0	1/4	1/4	1/4
	G:	1	1	1	0		1/4	1/4	1/4	0
	T:	1	1	1	2		1/4	1/4	1/4	2/4

Laplace's Rule of Succession adds 1 to each element of COUNT(*Motifs*), updating the two matrices to the following:

COUNT(<i>Motifs</i>)	A:	2+1	1+1	1+1	1+1	PROFILE(<i>Motifs</i>)	3/8	2/8	2/8	2/8
	C:	0+1	1+1	1+1	1+1		1/8	2/8	2/8	2/8
	G:	1+1	1+1	1+1	0+1		2/8	2/8	2/8	1/8
	T:	1+1	1+1	1+1	2+1		2/8	2/8	2/8	3/8

Greedy Algorithm: Iteration 1

Motifs **ACCT**

COUNT(<i>Motifs</i>)	A:	1+1	0+1	0+1	0+1	PROFILE(<i>Motifs</i>)	2/5	1/5	1/5	1/5
	C:	0+1	1+1	1+1	0+1		1/5	2/5	2/5	1/5
	G:	0+1	0+1	0+1	0+1		1/5	1/5	1/5	1/5
	T:	0+1	0+1	0+1	1+1		1/5	1/5	1/5	2/5

We use this profile matrix to compute the probabilities of all 4-mers in the second string from *Dna*:

g ATG	ATGT	TGT c	GT ct	T ctg	ctgt	tgtc
$1/5^4$	$4/5^4$	$1/5^4$	$4/5^4$	$2/5^4$	$2/5^4$	$1/5^4$



Inspiring Excellence

Greedy Algorithm: Iteration 2

Motifs **ACCT**
 ATGT

COUNT(<i>Motifs</i>)	A: 2+1 0+1 0+1 0+1	PROFILE(<i>Motifs</i>)	3/6 1/6 1/6 1/6
	C: 0+1 1+1 1+1 0+1		1/6 2/6 2/6 1/6
	G: 0+1 0+1 1+1 0+1		1/6 1/6 2/6 1/6
	T: 0+1 1+1 0+1 2+1		1/6 2/6 1/6 3/6

We use this profile matrix to compute the probabilities of all 4-mers in the third string from *Dna*:

acg G	cg GC	g GCG	GCGT	CGT t	GT ta	T tag
$12/6^4$	$2/6^4$	$2/6^4$	$12/6^4$	$3/6^4$	$2/6^4$	$2/6^4$



Inspiring Excellence

Greedy Algorithm: Iteration 3

Motifs
 ACCT
 ATGT
 acgG

COUNT(<i>Motifs</i>)	A: 3+1 0+1 0+1 1+1	PROFILE(<i>Motifs</i>)	4/7 1/7 1/7 1/7
	C: 0+1 2+1 1+1 0+1		1/7 3/7 2/7 1/7
	G: 0+1 0+1 2+1 1+1		1/7 1/7 3/7 2/7
	T: 0+1 1+1 0+1 2+1		1/7 2/7 1/7 3/7

We use this profile matrix to compute probabilities of all 4-mers in the fourth string from *Dna*:

ccct	ccta	cta A	ta AC	a ACG	ACGA	CGA g
18/7 ⁴	3/7 ⁴	2/7 ⁴	1/7 ⁴	16/7 ⁴	36/7 ⁴	2/7 ⁴



Inspiring Excellence

Greedy Algorithm: Iteration 4

Motifs

ACCT
ATGT
acgG
ACGA

	A: 4+1 0+1 0+1 0+1		5/8 1/8 1/8 2/8
COUNT(<i>Motifs</i>)	C: 0+1 3+1 1+1 0+1	PROFILE(<i>Motifs</i>)	1/8 4/8 2/8 1/8
	G: 0+1 0+1 3+1 1+1		1/8 1/8 4/8 2/8
	T: 0+1 1+1 0+1 2+1		1/8 2/8 1/8 3/8

We now use this profile to compute the probabilities of all 4-mers in the fifth string in *Dna*:

cgtc	gtca	tcag	cagA	agAG	gAGG	AGGT
$1/8^4$	$8/8^4$	$8/8^4$	$8/8^4$	$10/8^4$	$8/8^4$	$60/8^4$



Inspiring Excellence

Greedy Algorithm: Consensus

- Find the (4,1)-motif **ACGT** implanted in the following strings Dna.

tt**ACCT**taac

g**ATGT**ctgtc

acg**GCGT**tag

cccta**ACGA**g

cgtcag**AGGT**

ACCT

ATGT

Motifs acg**G**

ACGA

AGGT

CONSENSUS(*Motifs*) **ACGT**



Inspiring Excellence

Further Improvement!

<i>Profile</i>	A:	4/5	0	0	1/5		ttaccttaac
	C:	0	3/5	1/5	0		gatgtctgtc
	G:	1/5	1/5	4/5	0	<i>Dna</i>	acggcgtag
	T:	0	1/5	0	4/5		ccctaacgag cgtcagaggt

Taking the *Profile*-most probable 4-mer from each row of *Dna* produces the following 4-mers (shown in red):

MOTIFS(*Profile*, *Dna*)

tt	acct	taac
ga	tgt	ctgtc
acg	gcgt	tag
cccta	acgag	
cgtcag	aggt	

- Why would we do this? Because our hope is that MOTIFS(PROFILE(Motifs), Dna) has a better score than the original collection of k-mers Motifs. We can then form the profile matrix of these k-mers, PROFILE(MOTIFS(PROFILE(Motifs), Dna)), and continue...



Inspiring Excellence

A Monte Carlo Algorithm

RANDOMIZEDMOTIFSEARCH(*Dna*, *k*, *t*)

randomly select *k*-mers *Motifs* = (*Motif*₁, ..., *Motif*_{*t*}) in each string from *Dna*

BestMotifs ← *Motifs*

while forever

Profile ← PROFILE(*Motifs*)

Motifs ← MOTIFS(*Profile*, *Dna*)

if SCORE(*Motifs*) < SCORE(*BestMotifs*)

BestMotifs ← *Motifs*

else

return *BestMotifs*

- Since a single run of RANDOMIZEDMOTIFSEARCH may generate a rather poor set of motifs, bioinformaticians usually run this algorithm thousands of times. On each run, they begin from a new randomly selected set of *k*-mers, selecting the best set of *k*-mers found in all these runs.



Inspiring Excellence

A Monte Carlo Algorithm

ttACCT**taac**
gAT**GTct**gtc
Dna **ccgG**CGTtag
c**acta**ACGAg
cgtcag**AGGT**

Below, we construct the profile matrix **PROFILE**(*Motifs*) of the chosen 4-mers.

<i>Motifs</i>				PROFILE(<i>Motifs</i>)				
t	a	a	c	A:	0.4	0.2	0.2	0.2
G	T	c	t	C:	0.2	0.4	0.2	0.2
c	c	g	G	G:	0.2	0.2	0.4	0.2
a	c	t	a	T:	0.2	0.2	0.2	0.4
A	G	G	T					

A Monte Carlo Algorithm

ttAC	tACC	ACCT	CCTt	CTta	Ttaa	taac
.0016	.0016	.0128	.0064	.0016	.0016	.0016
gATG	ATGT	TGTc	GTct	Tctg	ctgt	tgtc
.0016	.0128	.0016	.0032	.0032	.0032	.0016
ccgG	cgGC	gGCG	GCGT	CGTt	GTta	Ttag
.0064	.0036	.0016	.0128	.0032	.0016	.0016
cact	acta	ctaA	taAC	aACG	ACGA	CGAg
.0032	.0064	.0016	.0016	.0032	.0128	.0016
cgtc	gtca	tcag	cagA	agAG	gAGG	AGGT
.0016	.0016	.0016	.0032	.0032	.0032	.0128



Inspiring Excellence

Gibbs Sampling

- RANDOMIZEDMOTIFSEARCH may change all t strings in Motifs in a single iteration.
 - This strategy may prove reckless, since some correct motifs (captured in Motifs) may potentially be discarded at the next iteration.
- GIBBSAMPLER is a more cautious iterative algorithm that discards a single k -mer from the current set of motifs at each iteration and decides to either keep it or replace it with a new one.

```
ttaccttaac      ttaccttaac
gatatctgtc      gatatcgtc
acggcgttcg →  acggcgttcg
ccctaaagag      ccctaaagag
cgtcagaggg      cgtcagagg
```

RANDOMIZEDMOTIFSEARCH
(may change all k -mers in one step)

```
ttaccttaac      ttaccttaac
gatatctgtc      gatatcgtc
acggcgttcg →  acggcgttcg
ccctaaagag      ccctaaagag
cgtcagaggg      cgtcagaggg
```

GIBBSAMPLER
(changes one k -mer in one step)



Inspiring Excellence

Gibbs Sampling

```
GIBBSAMPLER(Dna, k, t, N)  
  randomly select k-mers Motifs = (Motif1, ..., Motift) in each string from Dna  
  BestMotifs ← Motifs  
  for j ← 1 to N  
    i ← RANDOM(t)  
    Profile ← profile matrix formed from all strings in Motifs except for Motifi  
    Motifi ← Profile-randomly generated k-mer in the i-th sequence  
    if SCORE(Motifs) < SCORE(BestMotifs)  
      BestMotifs ← Motifs  
  return BestMotifs
```

- What is the value of *k*?
- What if the nucleotide distribution is skewed?



Inspiring Excellence

Gibbs Sampling: Iteration 1

```
      ttACCTtaac      ttACCTtaac
      gATGTctgtc      gATGTctgtc
Dna  ccgGCGTtag  →  -----
      cactaACGAg      cactaACGAg
      cgtcagAGGT      cgtcagAGGT
```

This results in the following motif, count, and profile matrices.

```
      t a a c
Motifs G T c t
      a c t a
      A G G T
```

A: 2 1 1 1	PROFILE(Motifs)	A: 2/4 1/4 1/4 1/4
C: 0 1 1 1		C: 0 1/4 1/4 1/4
G: 1 1 1 0		G: 1/4 1/4 1/4 0
T: 1 1 1 2		T: 1/4 1/4 1/4 2/4



Inspiring Excellence

Gibbs Sampling: Iteration 2

ccgG	cgGC	gGCG	GCGT	CGTt	GTta	Ttag
0	0	0	1/128	0	1/256	0

Application of Laplace's Rule of Succession to the count matrix above yields the following updated count and profile matrices:

	A: 3 2 2 2		A: 3/8 2/8 2/8 2/8
COUNT(Motifs)	C: 1 2 2 2	PROFILE(Motifs)	C: 1/8 2/8 2/8 2/8
	G: 2 2 2 1		G: 2/8 2/8 2/8 1/8
	T: 2 2 2 3		T: 2/8 2/8 2/8 3/8

After adding pseudocounts, the 4-mer probabilities in the deleted string ccgGCGTtag are recomputed as follows:

ccgG	cgGC	gGCG	GCGT	CGTt	GTta	Ttag
4/8 ⁴	8/8 ⁴	8/8 ⁴	24/8 ⁴	12/8 ⁴	16/8 ⁴	8/8 ⁴

Since these probabilities sum to $C = 80/8^4$, our hypothetical seven-sided die is represented by the random number generator

$$\begin{aligned} & \text{RANDOM} \left(\frac{4}{80/8^4}, \frac{8}{80/8^4}, \frac{8}{80/8^4}, \frac{24}{80/8^4}, \frac{12}{80/8^4}, \frac{16}{80/8^4}, \frac{8}{80/8^4} \right) \\ &= \text{RANDOM} \left(\frac{4}{80}, \frac{8}{80}, \frac{8}{80}, \frac{24}{80}, \frac{12}{80}, \frac{16}{80}, \frac{8}{80} \right). \end{aligned}$$



Inspiring Excellence

Gibbs Sampling: Iteration 3

```

      ttACCTtaac      -----
      gATGTctgtc      gATGTctgtc
Dna  ccgGCGTtag  →  ccgGCGTtag
      cactaACGAg      cactaACGAg
      cgtcagAGGT      cgtcagAGGT
  
```

After constructing the motif and profile matrices, we obtain the following:

	G	T	c	t		A:	2/4	0	0	1/4
<i>Motifs</i>	G	C	G	T	PROFILE(<i>Motifs</i>)	C:	0	2/4	1/4	0
	a	c	t	a		G:	2/4	1/4	2/4	0
	A	G	G	T		T:	0	1/4	1/4	3/4

Note that the profile matrix looks more biased toward the implanted motif than the previous profile matrix did. We update the count and profile matrices with pseudo-counts:

COUNT(<i>Motifs</i>)	A:	3	1	1	2	PROFILE(<i>Motifs</i>)	A:	3/8	1/8	1/8	2/8
	C:	1	3	2	1		C:	1/8	3/8	2/8	1/8
	G:	3	2	3	1		G:	3/8	2/8	3/8	1/8
	T:	1	2	2	4		T:	1/8	2/8	2/8	4/8

Then, we compute the probabilities of all 4-mers in the deleted string ttACCTtaac:

ttAC	tACC	ACCT	CCTt	CTta	Ttaa	taac
2/8 ⁴	2/8 ⁴	72/8 ⁴	24/8 ⁴	8/8 ⁴	4/8 ⁴	1/8 ⁴



Inspiring Excellence

Gibbs Sampling: Iteration 4

Dna tt**ACCT**taac tt**ACCT**taac
 gAT**GTct**gtc gAT**GTct**gtc
 ccg**GCGT**tag ccg**GCGT**tag
 c**acta**ACGAg -----
 cgtcag**AGGT** cgtcag**AGGT**

We further add pseudocounts and construct the resulting count and profile matrices:

		A	C	C	T	
<i>Motifs</i>		G	T	c	t	
		G	C	G	T	
		A	G	G	T	
	A:	3	1	1	1	A: 3/8 1/8 1/8 1/8
COUNT(<i>Motifs</i>)	C:	1	3	3	1	C: 1/8 3/8 3/8 1/8
	G:	3	2	3	1	G: 3/8 2/8 3/8 1/8
	T:	1	2	1	5	T: 1/8 2/8 1/8 5/8
		PROFILE(<i>Motifs</i>)				

We now compute the probabilities of all 4-mers in the deleted string cactaACGAg:

cact	acta	ctaA	taAC	aACG	ACGA	CGAg
$15/8^4$	$9/8^4$	$2/8^4$	$1/8^4$	$9/8^4$	$27/8^4$	$2/8^4$



Inspiring Excellence